

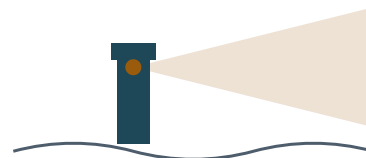
Lighting the Storm

An apprentice keeper repairs a failing tower, lets two shipwrecked sailors in from the storm, and crosses a worsening sea for the one part that can't be fixed on site, before a passenger ship is lost on the rocks.

By **Paul** Updated **25 September 2026**

First-person · single-player · PC (Steam)

Bracken Isle Light · Fl(2) W 15s



PART 1

Executive Summary

A 30-minute first-person atmospheric adventure about keeping a light alive through one storm, built by a solo developer with AI assistance. A 10-minute prototype comes first, over six weeks.

Pitch and player promise

You are the new apprentice to Ted, keeper of a remote, storm-battered lighthouse on Bracken Isle. You work through the building's failing systems under his direction, but the beacon itself won't stop flickering. A rowboat is driven onto Bracken Isle by the weak light; you watch it crash from the window, then open the lighthouse gate so the two survivors, Jeff and Daniel, can get inside. With Jeff beside you, you cross the storm in an engine boat to fetch a replacement burner from an abandoned outpost, then race back to fit it before a passenger ship reaches the rocks.

The central player skill is steering a powered boat past rocks and other hazards while the storm worsens. The experience should deliver sustained dread, hands-on mechanical work, and a consequential ending. There is no combat, no monster reveal and no jump scares. Folk-horror details invite interpretation without ever confirming a supernatural cause.

Design pillars

Every feature and story beat is judged against four pillars. Each has a test question used in reviews and cuts.

Pillar 1

The light must return

Everything leads back to the beacon. It is what you repair, it guides you home, and it decides whether the ship lives.

Test: does this connect to getting the light back, or to what happens if it fails?

Pillar 2

The storm sets the clock

Pressure comes from the world: the approaching ship's lights, worsening weather, falling visibility and rocks in the dark. Never a timer, meter or enemy.

Test: can the player feel time running out without any UI?

Pillar 3

Your hands, your eyes

Everything is first-person and physical. You fix things by hand, watch the crash through the window, and see the ship's fate from where you stand. Guidance comes from Ted and Jeff, not markers.

Test: is this happening to the player in the world, or being shown to them?

Pillar 4

Dread you can read

Sustained unease and ambiguity, never jump scares or a confirmed monster. The player always knows what to do and why they failed: one steering control, clear hazards, fair and recoverable mistakes.

Test: does this add unease without adding confusion?

Scope rule: small world, read closely. One lighthouse, one stretch of sea, one abandoned outpost, and no new major locations. A feature that fails a pillar's test, or expands the world without deepening the main loop, is deferred.

Cast

- **The Apprentice (player).** Characterised through action and a handwritten objective journal rather than dialogue trees.
- **Ted, the keeper.** Late 50s, thirty years in the tower. Gruff and blunt, with a guarded sense of care. He directs the repairs out loud from elsewhere in the building.
- **Jeff, the older seaman.** Hard, coarse and unhappy about helping, but his instinct is to keep people alive. He is the player's companion on both crossings and calls out navigation warnings.
- **Daniel, the younger seaman.** Shaken by the crash and just glad to be alive. He stays at the lighthouse.

Scope

	First playable prototype	Full game
Length	About 10 minutes	About 30 minutes
Content	One repair, crash seen from the window, gate, boarding with Jeff, outbound crossing, burner pickup, return crossing, burner fitting, win and fail outcomes	Full repair set, crash and gate, two crossings, outpost exploration, optional discoveries, win and fail outcomes
Art and audio	Greybox art, placeholder audio	Final art, audio and dialogue
Schedule	6 weeks, solo, AI-assisted	A further 5–6 months (estimate)
Purpose	Prove that hazards are readable and steering is enjoyable	Commercial release on Steam, with controller support

STRETCH GOAL

Wave hazards that push or roll the boat are a stretch goal for both the prototype and the full game. The crossings are designed to work fully without them. Waves are added only after steering passes its week-2 go/no-go test and only if the schedule has room, and they are the first thing cut if it slips.

Outcome checks

Prototype

- A new player understands what to repair, where to head, and which hazard to avoid, with no timer UI.
- The boat has one steering control, one fixed speed, clear feedback for near misses and collisions, and a recoverable error state.
- The ship’s lights and position, seen in first person, communicate the stakes.
- Both the win and fail outcomes can be reached.

Full game

- A first-time playthrough lasts about 30 minutes.
- The final game preserves unexplained environmental wrongness.

Game Mechanics

What the player does, in the order they do it, and the systems that shape each action.

Core loop

The loop mirrors one worsening night on duty and plays once through the main story.

- 01 **Repair the lighthouse.** Ted barks which system is broken and roughly where its part is. The player explores, diagnoses and fixes it with a fixed toolkit. The beacon alone keeps flickering. NO DEADLINE

- 02 **Watch the crash, open the gate.** Ted calls the player to the window, where a rowboat crashes onto Bracken Isle. He tells the player to open the lighthouse gate; Jeff and Daniel make their own way inside. Ted explains the burner must be fetched from the abandoned outpost. NO DEADLINE

- 03 **Board the engine boat with Jeff.** Boarding starts the hidden passenger-ship deadline, and Jeff spells out how much trouble they are in. DEADLINE STARTS

- 04 **Cross to the outpost.** Steer past rocks, buoys and debris in the storm while Jeff shouts directions.

- 05 **Pick up the burner.** Find it at the outpost and get it aboard.

- 06 **Cross back.** The storm peaks: denser hazards, lower visibility, fragile cargo aboard.

- 07 **Fit the burner.** Climb to the lamp room and install it as the ship's lights close on the rocks.

- 08 **Win or fail.** Beacon lit in time: the ship sails calmly into port. Deadline missed: the ship runs aground. OUTCOME

Lighthouse repairs

The lighthouse runs on interdependent systems: the lamp and optic assembly, the clockwork rotation mechanism, the fuel and fog-signal lines, and stormproofing (shutters and seals). Each broken system is fixed through a short, first-person puzzle, such as rewiring a junction box, swapping a gear or patching a leak, using a small fixed toolkit rather than a crafting menu. The prototype includes one repair; the full game includes the full set.

- **Ted's guidance** is the first assist mechanism. He names the part and roughly where to find it; how specific he is scales with the difficulty preset.

- **The journal** is a handwritten-style checklist of what is broken, updated as systems are diagnosed. It reflects progress but never controls it.
- **Gating.** Some repairs open new areas, for example repairing the winch opens the hatch to the boat dock. This gives one building light Metroidvania-style gating without a large map.
- **The beacon as inciting incident.** Once every other system works, the beacon still flickers. That unresolved fault draws Ted to the window and triggers the crash, so a mechanical dead end becomes the story's turn instead of a quest marker.

The crash and the gate

The crash is scripted and seen entirely in first person through the lighthouse window. It triggers when the window is in view, with a short fallback so it cannot be missed. Opening the gate is a single interaction; Jeff and Daniel walk in on their own, with no escort or rescue. Opening the gate advances the story and opens the route to the engine boat, where Jeff waits.

Sea crossing and boat navigation

The player pilots a small engine boat at **one fixed speed with one control: steering**. There is no throttle and no oars. The challenge is steering past placed hazards (rocks, reefs, buoys and floating debris) along a route that narrows and darkens as the storm worsens. The sea surface moves for atmosphere but does not push the boat.

- **Jeff's calls** are the second assist mechanism. His warnings read from the same hazard data as the world, so they never disagree with the real threat. How early and how often he calls scales with the difficulty preset.
- **Navigation aids:** a compass, the lighthouse beam as a guiding light when heading home, and silhouettes of buoys and rocks in low visibility.
- **Risk states:** a collision takes on water, slows the boat, or causes a brief capsized and recovery at a nearby safe point, all costing time. On the return leg, rough handling can knock the burner loose, costing time to recover it. The burner is never permanently lost (proposed).
- **Stretch goal:** authored waves that push and roll the boat, added as an optional layer only if time allows.

Tension and the deadline

The central pressure is a countdown the player never sees as a number. It starts only when the player boards the boat; everything before that plays at its own pace. From then on, the approaching ship's lights on the horizon and the rising storm (wind volume, rain density, sea swell, visibility) communicate how much time is left. It is the only timer in the game: the same deadline runs through both crossings, the pickup and the final repair.

Most failures are soft. A collision costs time rather than ending the run, which keeps the player in the fiction. The exception is the deadline itself, which is a true fail state, so the ending means something.

Escalation curve

Act 1 · Lighthouse Calm, tutorialised, interrupted by the crash	Act 2 · Outbound Moderate hazards, deadline running	Act 3 · Return Severe: the hardest skill test in the game	Climax · Burner A multi-step repair under the same deadline
---	---	---	---

Difficulty and accessibility

Difficulty is mainly a pacing and tension curve, not a skill gate. A single preset controls how specific Ted’s directions are, how much Jeff helps on the water, and how intense the hazards are, without changing the story or outcomes. Navigation calls are subtitled with speaker names and an optional visual direction cue. Steering and interaction are remappable, with adjustable sensitivity and a reduced-hazard preset.

Presentation

All viewpoints are first-person, with no cutaway or exterior cameras. Warm oil-lamp light indoors plays against blue-grey storm light outside, and the beacon’s sweeping beam is the constant visual anchor. Sound is diegetic first: wind, rain, groaning timber, machinery, buoy bells and a distant foghorn, with no shock stingers. Strange symbols, unexplained shrines and architecture that doesn’t quite fit suggest a history without ever explaining it.

AI Architecture

The game is built by one developer working with a team of AI agents. Each agent owns one department of the build. Below, each is described through what the player experiences as a result of its work.

SCOPE OF AI USE

AI is used only during development. The shipped game makes no AI calls at runtime: Ted's and Jeff's lines, the storm and the ship are authored and scripted, so the game has no API dependency or per-player running cost.

Principles

- **One agent, one department.** Each agent works from a bounded task brief naming its goal, inputs, owned files, interfaces and acceptance checks.
- **The developer owns judgement.** Steering feel, tone, art fit, dialogue performance and story interpretation are human decisions. AI output is a first pass that the developer reviews and merges.
- **The pillars bind every agent.** Work that fails a pillar's test is returned, whatever its technical quality.
- **No silent canon.** When a story or design choice is unresolved, the agent stops and asks instead of deciding.

The agents and their effect on play

Boat / Physics agent Serves pillar 4 · the core skill

WHAT THE PLAYER EXPERIENCES

The feel of every crossing: how quickly the boat answers the stick, how clearly a near miss or collision registers, and how quickly the player recovers and gets back on course.

WHAT IT BUILDS

- Boat controller at one fixed speed
- Hazard collisions and recovery
- Tuning values held as data
- Stretch: wave layer behind an on/off setting

Review gate: the developer plays the week-2 steering spike and makes the go/no-go call on feel.

Level Layout agent Serves pillar 3 and the scope rule

WHAT THE PLAYER EXPERIENCES

Where they can go and what they can see: the climb through the tower, the window that frames the crash, the gate, a sea route that narrows toward the outpost, and sightlines to the beacon and the ship.

WHAT IT BUILDS

- Greybox lighthouse, sea corridor and outpost
- Interaction anchors for other agents
- Sightline checks for the crash and ending

Review gate: traversal and sightline checks in the engine.

Gameplay / Systems agent

Serves pillars 1 and 2

WHAT THE PLAYER EXPERIENCES

The rules of the night: the story advancing in the right order, the deadline starting the moment they board, fair time penalties for mistakes, and an ending that resolves once as a win or a fail.

WHAT IT BUILDS

- Phase controller and save state
- Repair jobs from data
- Crash trigger, gate and boarding
- Hidden deadline, cargo, win and fail

Review gate: the 10-minute slice plays end to end with no dead ends.

Lighting agent

Serves pillars 1, 2 and 4

WHAT THE PLAYER EXPERIENCES

The beacon as both objective and guide home, warm lamplight against the storm, and darkness that deepens as time runs out while hazards stay readable.

WHAT IT BUILDS

- Beacon sweep and dynamic lights
- Storm visibility presets by phase
- Minimum hazard-visibility rules

Review gate: the beacon and hazards stay legible at every phase.

Sound agent Serves pillars 2 and 4

WHAT THE PLAYER EXPERIENCES

A storm they can hear getting worse, which tells them time is short without a timer, and Jeff's voice cutting through it when they need a warning. No shock stingers.

WHAT IT BUILDS

- Layered wind, rain, machinery and foghorn
- State-based mix and escalation
- Source list and licences

Review gate: Jeff's warnings remain audible at peak storm.

Asset agent Serves pillars 3 and 4

WHAT THE PLAYER EXPERIENCES

The physical objects they handle during repairs, and the unsettling details (symbols, shrines, out-of-date posters) that make the lighthouse feel wrong.

WHAT IT BUILDS

- First-pass props and set dressing
- Source meshes and import notes
- Provenance records for every asset

Review gate: assets import cleanly and fit the approved reference sheet.

Dialogue agent Serves pillars 3 and 4

WHAT THE PLAYER EXPERIENCES

Ted's directions during repairs, Jeff's calls on the water, and the journal's objectives. Together these are how the player knows what to do without markers, and they get sparser on harder settings.

WHAT IT BUILDS

- Lines for Ted, Jeff and Daniel as data
- Subtitle and difficulty variants
- Journal text

Review gate: the developer approves voice and lore before anything is recorded.

QA agent Serves all pillars

WHAT THE PLAYER EXPERIENCES

A night with no dead ends: every phase finishes, both outcomes can be reached, and mistakes are recoverable.

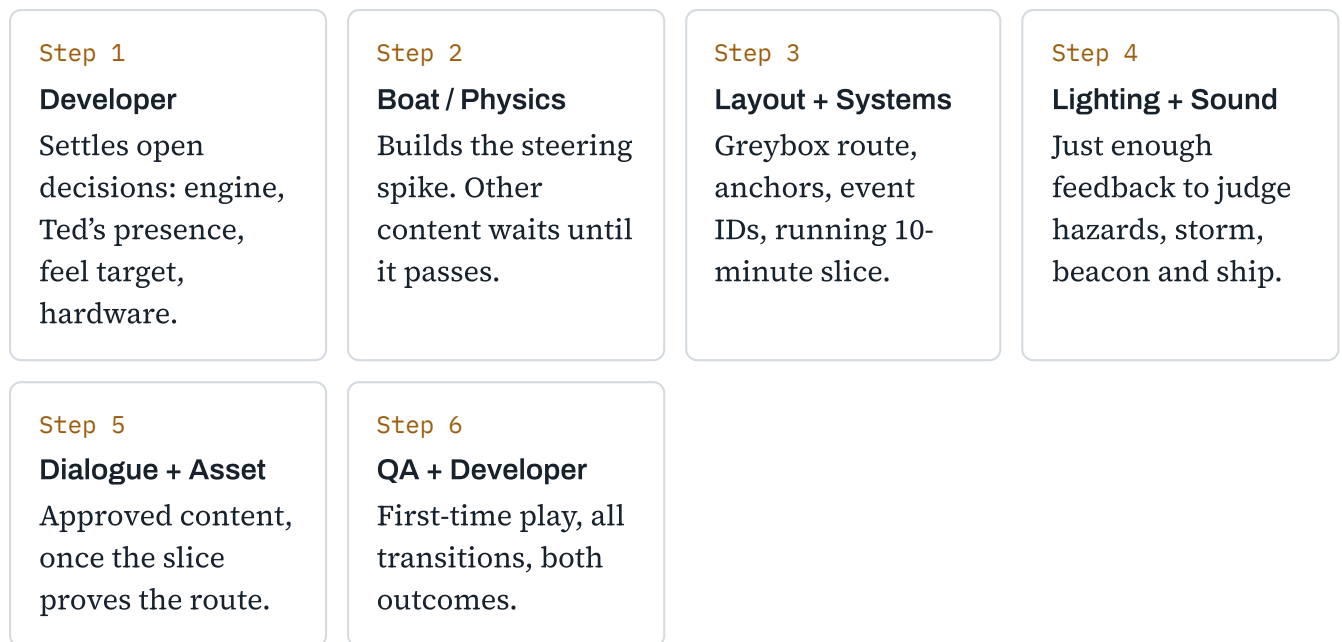
WHAT IT BUILDS

- Reproducible scenario list
- Bug reports against acceptance checks
- Checks with waves on and off

Review gate: the developer reproduces and prioritises each finding.

These are roles, not eight agents running at once. One agent can take successive bounded tasks.

Build sequence



Handoff review

Every handoff returns editable source, licences for any third-party material, test instructions and known limitations, and is checked against the same questions:

- Does it run in the integrated project from a clean checkout?
- Are state changes and event IDs consistent with the shared contract?
- Does it pass its task's acceptance checks?
- Does it pass each pillar's test and the scope rule, without deciding an unresolved story point?
- What did human playtesting show about feel, clarity and pacing?

Technical Strategy

Engine, build contract and schedule; how the agent roles are organised; the token budget; and the API and plan constraints the work runs under.

Engine

Proposed: Godot 4 with GDScript. It is free with no royalties, iterates quickly on modest hardware, and stores scenes as plain text, so AI agents can read and edit scenes as well as scripts. Its main weakness, no built-in ocean, is off the critical path now that waves are a stretch goal.

Factor	Godot 4	Unity	Unreal 5	Phaser
First-person 3D	Good	Very good	Excellent	Not supported (2D)
Steering past hazards	Straightforward	Straightforward	Straightforward	2D only
Waves (stretch)	Build yourself	Asset Store packages	Built-in, realism-focused	2D only
Storm lighting	Adequate with effort	Very good	Excellent	Poor
AI can edit scenes	Yes (text)	Partly (YAML)	No (binary Blueprints)	Yes (all code)
Iteration speed	Fast	Medium	Slow	Fastest
Licence	Free, MIT	Free below revenue threshold	Royalty above high threshold	Free, MIT

Unity is the alternative if the developer already knows C#. Unreal is ruled out because Blueprints shut out AI help and iteration is slow. Phaser cannot meet the first-person brief (pillar 3).

Build contract

- **Separately owned scenes:** lighthouse, sea, outpost, shared player and interaction scenes, and one central game-state controller.
- **Content as data** with stable IDs: repairs, dialogue, assist presets and journal text.
- **Explicit phases:** LIGHTHOUSE_REPAIRS → CRASH_AND_GATE → BOARDING → OUTBOUND → OUTPOST → RETURN → FINAL_REPAIR → RESOLUTION (WIN | FAIL)
- **Invariants:** the deadline is inactive before boarding; Jeff is the only companion; pickup precedes return; the outcome resolves exactly once.

- **Events, not side effects:** audio, UI and dialogue respond to game events and never change progression directly.
- **Shared hazard forecast:** the hazards, the boat's reaction and Jeff's calls read from one source, so warnings always match the real threat.

Milestones

Gate	Deliverable	Acceptance
0 · Steering spike	Greybox boat, hazards, heading and collision feedback	Steering alone completes a short route; tunable without rewrites. Stop or redesign if not enjoyable.
1 · 10-minute slice	The full prototype path with win and fail outcomes	Playable end to end; deadline starts only on boarding; mistakes cost time but never trap the run.
2 · Narrative alpha	All repairs, outpost discoveries, full dialogue	Every story state can be finished; no dead-end saves or lost cargo.
3 · Content and polish	Final art, audio, accessibility, checkpoints	New players show legible navigation and 30-minute pacing.

Six-week prototype schedule

Week	Focus	Exit test
1	Project setup and pipeline; boat at fixed speed with one steering axis; greybox sea; first rocks, buoys and debris	The boat steers and collides reliably
2	Steering feel, hazard readability, collision feedback and recovery (Gate 0)	Go/no-go: steering past hazards is fun on its own
3	Outpost and burner pickup; harder return crossing; storm visibility and lighting	Both crossings playable end to end
4	One repair, crash through the window, gate, Jeff at the boat, Jeff's calls	The full 10-minute path is playable
5	Hidden deadline, win and fail outcomes, placeholder audio	Both outcomes reachable
6	Playtests with 3–5 new players, tuning, bug fixing, tester build (Gate 1)	Prototype outcome checks pass

Waves are not in this schedule. They start only after Gate 0 passes and the current week's exit test is met (earliest: spare time in week 5), sit behind one on/off setting, and are cut first if anything slips.

Agent roles and working rules

The eight roles in Part 3 map onto the build contract as follows. Boat/Physics and Gameplay/Systems agree event names before working in parallel; Level Layout supplies scene anchors before Lighting, Sound and Asset integrate.

Agent	Owns	Input contract
Boat / Physics	Boat scene, controller, tuning resource	Hazard forecast API, one steering axis, recovery event
Level Layout	Greybox scenes, anchors	Location sizes, beacon visibility, interaction anchors
Gameplay / Systems	Phase controller, save state, repairs, deadline	Stable event and save IDs
Lighting	Light rigs, visibility presets	Scene anchors, performance budget, visibility rules
Sound	Audio banks, mix states	Escalation events, speaker priority
Asset	Source meshes, import settings	Scale, pivot, collision, palette, reference sheet
Dialogue	Dialogue and journal data	Character voices, phase events, ambiguity rule
QA	Scenario list, bug reports	Playable build, event log, acceptance checklist

- One branch or change set per task; no agent overwrites another's scene, script or asset.
- Contract changes are raised before any dependent file is edited.
- Each task runs the project build and focused checks, and reports what was observed, what could not be verified, and which human decision remains.

Token budget

Estimated from a unit of one **agent task**: about 40 model turns at an average 60K-token context, giving about 2.4M input tokens (about 90% cache reads) and 40K output tokens. Full-game tasks after the prototype assume a larger 90K context as the codebase grows.

~\$500

Prototype · ~250 tasks ·
range \$250–\$1,400

~\$2,050

Full game incl.
prototype · ~835 tasks ·
range \$1,000–\$5,000

~20M

Input tokens per
working day during the
prototype, mostly
cached

Model	Role in the mix	Input / MTok	Output / MTok	Cache read	Per task
Claude Opus 5.5	Implementation and debugging (70%)	\$4.00	\$20.00	\$0.20	\$2.36
Claude Sonnet 5	Routine scripting, data, docs (20%)	\$2.00	\$10.00	\$0.20	\$1.40
Claude Haiku 4.5	Search, file sweeps, formatting (10%)	\$1.00	\$5.00	\$0.10	\$0.70
Claude Fable 5.1	Not planned; comparison only	\$10.00	\$50.00	\$0.25	\$5.36

Prototype workstream	Tasks
Planning and docs	15
Project setup and asset pipeline	10
Boat, hazards, steering and tuning	40
Stretch: wave hazards, only if time allows	15
Greybox levels and sightlines	20
Interaction, repair, crash and gate	30
Jeff’s calls and placeholder dialogue	20
State machine, deadline, endings and save	20
Lighting, weather and audio integration	30
Bug fixing and integration	40

Prototype workstream	Tasks
Playtest analysis and tuning	10
Total	250

Not included: AI image, 3D, music or voice tools billed by other vendors; paid engine assets; the Steam Direct app fee. Real usage will be measured after week 1 and the forecast revised.

API and plan constraints

Constraint	Effect on the project
Development-time only	No runtime AI in the shipped game, so no API keys, latency, per-player cost or availability risk after release.
Plan usage limits	Development runs on a Claude Team plan with rolling 5-hour and weekly limits. Extra usage is off, so hitting a limit pauses work until it resets. The prototype pace is estimated at about 30–50% of the weekly limit; this is checked at the end of week 1.
Context windows	Opus 5.5 and Sonnet 5 accept up to 1M tokens; Haiku 4.5 accepts 200K. Narrow task briefs keep contexts far below this, which also keeps cost down.
Prompt caching	Caching matches on the exact prefix; any change to earlier content invalidates everything after it. Stable project docs and briefs make most input cheap cache reads (writes cost about 1.25× input).
Opus 5.5 behaviour	Thinking cannot be switched off; depth and spend are controlled with the effort setting, which defaults to medium. Routine tasks run at lower effort.
Cost levers	Route routine work to Sonnet 5 or Haiku 4.5, keep contexts small, and start fresh sessions between unrelated tasks.
Asset rights	Every AI-generated or third-party asset carries a provenance and licence record before it is merged.

Feasibility

Target	6 weeks, solo, AI-assisted	Reasoning
10-minute prototype	Feasible, with conditions	A greybox build for playtesters, not a product to sell. It depends on steering passing in week 2, frozen scope, placeholder art and audio, and week 6 kept for playtests.

Target	6 weeks, solo, AI-assisted	Reasoning
30-minute full game	Not feasible	Estimated 27–31 weeks solo (about 5 months with stock assets and AI voice). The largest gaps are 3D art and character animation, QA, and Steam launch preparation, including a public Coming Soon page for at least two weeks before release.

AI realistically speeds up code, data, docs and dialogue drafts by 1.3–2×. It helps little with steering feel, visual readability, consistent 3D art and playtesting. The plan therefore commits the six weeks to the prototype, and uses the week-2 steering result and week-6 playtests to decide whether to fund the full game.

Risks

1. **Boat feel** is the largest risk. Making waves a stretch goal removes its hardest part from the critical path, and Gate 0 tests it first.
2. **Scope creep** across repairs, sequences and environmental detail, held back by the pillars and the scope rule.
3. **Usage limits** interrupting work, monitored weekly and handled by routing routine tasks to smaller models.

Open decisions

Items still to be settled by the developer. Proposed answers are in use until confirmed.

Area	Decision	Current proposal
Story	Ted's presence	Ted is present and speaks live; recordings belong to earlier keepers
Story	Daniel after the gate	Open: later role, or stays out of sight
Deadline	Length	Set from playtests, with room for some mistakes
Deadline	Pause and checkpoints	Stops in menus and pause; time saved at checkpoints
Deadline	After the fail state	Open: checkpoint retry, restart, or end with retry option
Boat	Crossing failure policy	Collisions cost time; burner never permanently lost
Boat	Bailing interaction	Only if playtests show it helps
Production	Engine	Godot 4
Production	Target hardware and feel target	To define before Gate 0
Production	Controller support in the prototype	Open
Production	Which repair is in the prototype	Open
Stretch	Wave hazards	Decide at the end of week 4

Beyond scope

Ideas deliberately kept out of the prototype and full game. None is scheduled; each would need to pass the pillars first.

- **Companion choice.** Choose Jeff or Daniel before boarding; either can complete both crossings.
- **Daniel as a companion.** A flashlight that is faster to read than Jeff's voice but occasionally wrong. It conflicts with pillar 4, so it would need playtesting first.
- **Hidden coda.** An ending unlocked by journal fragments, shrines and symbols, revealing what happened to earlier keepers while leaving the supernatural unconfirmed.
- **Full Gale mode.** A New Game+ run that remixes the repairs and crossings into a harder, repeatable shift.